

Testing and debugging functions

Project: Python readiness portfolio

1. Your mission

Your learning target

I can design revealing tests for a function.

I can compare expected and actual results.

I can repair a defect and rerun earlier cases.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Reading structured data. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Return values and local state. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Design revealing tests for a function

Compare expected and actual results

Repair a defect and rerun earlier cases

Retrieve what you already know

For capacity 9 and used 2, what should remaining return?

My starting explanation

This week's end goal

Trace testing and debugging functions and explain the changed program before running it.

2. Notice the evidence

A function can pass one example by coincidence. Start with a written rule, calculate expected results independently, and include ordinary, zero and boundary cases. We will test a subtraction function under the contract $0 \leq \text{used} \leq \text{capacity}$.

List the given facts and one fact that is not supplied.

3. See how it works

```
def remaining(capacity, used):  
    return capacity - used  
  
cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]  
for capacity, used, expected in cases:  
    actual = remaining(capacity, used)  
    print(actual == expected)
```

Follow the evidence

Step or case · Evidence · Meaning

Step or case: Ordinary case

Evidence: 8 and 3

Meaning: Expect 5

Step or case: Full case

Evidence: 8 and 8

Meaning: Expect 0

Step or case: Unused case

Evidence: 8 and 0

Meaning: Expect 8

Read each row from left to right and connect the evidence to the stated rule.

Each parenthesized group is a tuple, an ordered group of values. The loop unpacks each group into three names. Equality compares the actual result to the independently calculated expected result. The worked script prints True three times. An assert actual == expected could instead stop at the first mismatch with AssertionError.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

A faulty function always returns 4. Which of your three cases pass and fail?

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Remove the test at 3 and keep only 2 and 4.

Prediction: The buggy <= function passes those remaining tests.

Reason: Both procedures agree away from the equality boundary.

5. Words that unlock the idea

Word · Meaning

Word: Test case

Meaning: Inputs and an expected result

Word: Boundary

Meaning: An edge of a permitted range

Word: Regression

Meaning: A previously working case that fails after a change

Word: Assertion

Meaning: A check that raises an error when its condition is false

Use two terms to explain a step in this week's example.

6. Follow a worked example

Would replacing the body with return 5 pass every worked case? Identify the first revealing test.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Testing and debugging functions

Use a test that catches the boundary bug. A function should classify an integer as tiny exactly when it is less than three.

The facts we will use

Python 3 intended function: `return x < 3`. Test cases: `(2, True)`, `(3, False)`, `(4, False)`. Candidate bug: `return x <= 3`.

Testing and debugging functions

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Record contract

Strict less-than excludes the value 3.



2. Run boundary cases

The intended outputs are True, False, False for 2,3,4.



3. Locate the bug

Using `<=` passes 2 but fails the test at 3 before the later test.

CHANGE ONE THING

Remove the test at 3 and keep only 2 and 4.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Record contract

What happens and why: Strict less-than excludes the value 3.

Step: 2. Run boundary cases

What happens and why: The intended outputs are True, False, False for 2,3,4.

Step: 3. Locate the bug

What happens and why: Using $\leq$ passes 2 but fails the test at 3 before the later test.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

A revealing test distinguishes a correct implementation from a plausible mistake. Checking only values far from the boundary would miss this inclusive-versus-strict bug.

What this example does not establish

Passing these cases does not prove correctness for every type or input; the contract and coverage determine what was actually checked.

7. Find and repair a mistake

A faulty `return capacity + used` gives 11, 16 and 8. The unused case passes even though the rule is wrong. Repair the operator to subtraction and rerun all three cases. Never change an expected result merely to match faulty output.

Locate the earliest unsupported step

A faulty function adds capacity and used. For (6, 2), state actual and expected. Repair it and name a regression case.

State one repair and the checks you would rerun.

8. Practise together

Give expected results for `remaining(6, 2)`, `remaining(6, 6)` and `remaining(6, 0)`.

Would replacing the body with `return 5` pass every worked case? Identify the first revealing test.

Explain your prediction to a partner before comparing results.

Practise with less help: Find a revealing case

Use workbook W2 for the connected example “Testing and debugging functions”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Testing and debugging functions. Proposed explanation: “A successful run proves the conclusion”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Predict the three correct outputs. Which test first catches $\leq$, and why do tests at only two and four miss

the bug?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

A faulty function always returns 4. Which of your three cases pass and fail?

```
def remaining(capacity, used):  
    return capacity - used  
  
cases = [(8, 3, 5), (8, 8, 0), (8, 0, 8)]  
  
for capacity, used, expected in cases:  
    actual = remaining(capacity, used)  
    print(actual == expected)
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Give expected results for `remaining(6, 2)`, `remaining(6, 6)` and `remaining(6, 0)`.

A faulty function adds capacity and used. For `(6, 2)`, state actual and expected. Repair it and name a regression case.

All three stock tests pass. Does that prove correct handling of negative counts?

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled testing and debugging functions evidence sheet to Python readiness portfolio. Show how you can design revealing tests for a function. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week’s record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

12. Show what you know

Passing this small suite supports these particular cases. It neither proves every allowed input correct nor checks invalid input handling. A test outside the contract needs a separately specified expected behavior.

All three stock tests pass. Does that prove correct handling of negative counts?

All three stock tests pass. Does that prove correct handling of negative counts?

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

Follow the evidence

Step or case · Evidence · Meaning

Step or case: Ordinary case

Evidence: 8 and 3

Meaning: Expect 5

Step or case: Full case

Evidence: 8 and 8

Meaning: Expect 0

Step or case: Unused case

Evidence: 8 and 0

Meaning: Expect 8

Read each row from left to right and connect the evidence to the stated rule.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Use boundary tests to reveal a function defect

Predict the output. If the function mistakenly uses $x \leq 10$, identify the first failing assertion and whether the final print happens. State the repair and which earlier tests must be rerun. Do these cases prove all input types are handled?

```
def below_limit(x):  
    return x < 10  
  
assert below_limit(9) is True  
assert below_limit(10) is False  
assert below_limit(11) is False  
print("three checks passed")
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Define `stock(packs, loose)` to return `packs * 4 + loose`. Create expected tests for (2, 3), (0, 3), (2, 0), then show which detects a faulty `packs + loose` body.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.